Dokumentasi Jobsheet 4 — UI/UX Design

Dokumentasi ini melanjutkan dokumentasi jobsheet-03, tapi isinya **berbeda bentuk** dari dokumentasi jobsheet sebelumnya. Kalau jobsheet-01 sampai jobsheet-03 dokumentasinya membedah kode HTML/CSS baris per baris, jobsheet-04 **tidak menambah kode apa pun** — sesuai README.md jobsheet ini: *"Tidak ada perubahan kode — halaman HTML/CSS tetap sama persis dengan Jobsheet 3."*

Jadi dokumentasi ini fokus ke hal yang benar-benar baru: **proses** perancangan (UI/UX design)** yang dituangkan di `docs/wireframe.md` —** sebuah dokumen rencana untuk fitur yang **belum ada kodenya** (Login, Dashboard Petugas, Peminjaman, Pengembalian, Riwayat), yang baru akan mulai diimplementasikan di jobsheet-jobsheet berikutnya.

Kenapa Ada Jobsheet Tanpa Kode Baru?

Ini bukan kekurangan — justru ini bagian penting dari cara kerja pengembangan software yang benar. Sebelum menulis kode untuk fitur baru yang cukup kompleks (Login, transaksi Peminjaman/Pengembalian), developer terlebih dulu **merancang** bagaimana fitur itu akan terlihat dan bagaimana alur penggunaannya — supaya tidak menulis kode dulu baru menyadari alurnya membingungkan atau tidak lengkap. Proses merancang ini disebut **UI/UX design**, dan dituangkan dalam bentuk **wireframe** dan **user flow** — dua istilah yang akan dijelaskan tuntas di bab 2 dan 3.

Daftar Isi

1. Konsep Dasar UI/UX Design
2. Cara Membaca Wireframe
3. User Flow: Peminjaman & Pengembalian
4. Aktor & Kaitannya dengan Otorisasi
5. Keterhubungan dengan Kode yang Sudah Ada
6. Rangkuman & Latihan Lanjutan

Struktur Folder

```
jobsheet-04/
├── index.html              # Sama persis dengan jobsheet-03
├── assets/css/style.css    # Sama persis dengan jobsheet-03
```

```
├── buku/                  # Sama persis dengan jobsheet-03
├── anggota/               # Sama persis dengan jobsheet-03
├── docs/
│   └── wireframe.md       # BARU — rancangan fitur yang belum dikoding
├── Infografis.png
├── README.md
└── Dokumentasi/           # Folder dokumentasi ini
```

Karena HTML/CSS-nya identik dengan jobsheet-03, kamu tidak perlu membaca ulang penjelasan tag/CSS — cukup rujuk kembali ke dokumentasi jobsheet-01, jobsheet-02, dan jobsheet-03 kalau perlu menyegarkan ingatan.

1. Konsep Dasar UI/UX Design

1.1 Apa Beda UI dan UX?

Dua istilah ini sering disebut berbarengan, tapi artinya berbeda:

Istilah	Kepanjangan	Fokus
UI	*User Interface* (Antarmuka Pengguna)	**Tampilan** — warna, tombol, tata letak, tipografi. Ini yang sudah kamu bangun sejak jobsheet-02 lewat `style.css` (lihat dokumentasinya).
UX	*User Experience* (Pengalaman Pengguna)	**Alur & rasa** — apakah pengguna mudah menyelesaikan tugasnya, apakah langkah-langkahnya masuk akal, apakah tidak membingungkan.

Analogi sederhana: kalau sebuah aplikasi adalah sebuah toko, **UI** adalah bagaimana rak dan etalase toko itu ditata supaya enak dipandang; **UX** adalah apakah pelanggan bisa dengan mudah menemukan barang yang dicari dan sampai ke kasir tanpa tersesat. Toko bisa saja terlihat cantik (UI bagus) tapi membingungkan untuk berbelanja (UX buruk), atau sebaliknya.

1.2 Kenapa Merancang Dulu, Baru Coding?

Bayangkan langsung menulis kode HTML untuk halaman "Peminjaman Buku" tanpa rancangan lebih dulu — kamu harus memutuskan sekaligus: field apa saja yang dibutuhkan, urutan

langkahnya bagaimana, apa yang terjadi kalau buku stoknya habis, halaman apa yang muncul setelah data disimpan, dan seterusnya — semuanya sambil menulis tag HTML. Sangat mudah lupa satu kondisi penting (seperti "buku stok 0 tidak boleh dipinjam") kalau merancang dan menulis kode dilakukan **bersamaan**.

Dengan membuat rancangan dulu di atas kertas/teks (seperti `docs/wireframe.md`), developer bisa:

1. Memikirkan **seluruh alur** dulu sebelum terikat detail teknis kode.
2. Menemukan celah/pertanyaan (misalnya: "bagaimana kalau anggota masih punya tunggakan?") **sebelum** menulis kode, saat masih murah untuk diubah — mengubah satu baris teks rancangan jauh lebih murah daripada menulis ulang kode yang sudah jadi.
3. Punya "peta" yang jelas untuk dirujuk saat mulai coding beneran nanti.

Inilah kenapa jobsheet-04 ini **sengaja tidak menambah kode** — seluruh energi jobsheet ini dicurahkan untuk merancang dulu, sesuai Sub-CPMK jobsheet ini: *"Merancang UI/UX aplikasi (proyek)."*

1.3 Dua Alat Bantu Rancangan: Wireframe & User Flow

`docs/wireframe.md` berisi dua jenis dokumen rancangan yang saling melengkapi:

- **Wireframe** — sketsa kasar tata letak *satu halaman*, tanpa detail warna/font, hanya menunjukkan elemen apa saja yang ada dan di mana posisinya. Dibahas detail di bab 2.
- **User flow** — diagram yang menunjukkan **urutan perpindahan** antar halaman/langkah yang dilalui pengguna untuk menyelesaikan satu tugas (misalnya meminjam buku). Dibahas detail di bab 3.

Kalau wireframe menjawab "halaman ini isinya apa saja?", user flow menjawab "untuk sampai ke sini, pengguna harus lewat mana saja?" — keduanya perlu dilihat bersamaan untuk memahami rancangan secara utuh.

Lanjut ke: Cara Membaca Wireframe

2. Cara Membaca Wireframe

Wireframe yang dipakai di `docs/wireframe.md` ditulis dalam bentuk **ASCII art** (gambar dari karakter teks biasa) di dalam blok kode markdown — mirip pendekatan yang tadinya dipakai untuk diagram box model di dokumentasi jobsheet-02 sebelum diganti SVG. Bedanya, untuk **wireframe rancangan** (bukan diagram final yang dipublikasikan), ASCII art justru pilihan yang tepat: cepat

digambar, mudah diubah lagi kalau rancangan berubah, dan tidak butuh alat desain khusus — cocok untuk tahap "sketsa kasar" sebelum sesuatu benar-benar dibangun.

2.1 Contoh: Wireframe Halaman Login

```
+-------------------------------------+
|             SIMPUS-Mini             |
|-------------------------------------|
|                                     |
|       [ Login Petugas ]             |
|                                     |
|   Username : [_____________]        |
|   Password : [_____________]        |
|                                     |
|          [   Masuk   ]              |
|                                     |
|   Belum punya akun? Daftar di sini  |
+-------------------------------------+
```

2.2 Aturan Membaca Simbol-Simbolnya

Wireframe teks seperti ini punya konvensi (kesepakatan) simbol yang sederhana:

- Kotak dari karakter pagar (`+`), strip (`-`), dan garis tegak lurus (dibaca "pipe") — menandai **batas luar** sebuah halaman atau kotak/panel di dalamnya.
- Garis tegak lurus tunggal di tengah kotak (garis vertikal pemisah) — menandai **pembatas antar bagian/section** di dalam satu halaman, mirip fungsi elemen `<section>` yang sudah kamu kenal dari dokumentasi jobsheet-01.
- `[_____________]` (kurung siku berisi garis bawah) — sebuah **kotak input**, tempat pengguna mengetik teks (nantinya jadi `<input type="text">` atau `<input type="password">`, ingat konsep ini dari dokumentasi jobsheet-01).
- `[ Teks ]` (kurung siku berisi teks) — sebuah **tombol** yang bisa diklik (nantinya jadi `<button>`).
- Teks biasa tanpa kurung — label atau keterangan statis (nantinya jadi `<label>`, `<h1>`, `<p>`, dst).

Menerjemahkan wireframe Login di atas ke istilah HTML yang sudah kamu kenal:

- `SIMPUS-Mini` di baris atas → nantinya jadi `<header><h1>` seperti di semua halaman yang sudah ada.

- `Login Petugas` → judul section, mirip `<h2>` di halaman-halaman lain (misalnya "Daftar Buku" di dokumentasi jobsheet-01).
- `Username : [_____]` dan `Password : [_____]` → akan jadi pasangan `<label>` + `<input>`, persis polanya seperti form "Tambah Buku" yang sudah kamu pelajari di dokumentasi jobsheet-01 §4.3 — hanya saja `Password` nantinya memakai `type="password"` (jenis input baru yang belum pernah dipakai di jobsheet-01 sampai jobsheet-03, supaya karakter yang diketik disembunyikan sebagai titik-titik).
- `[ Masuk ]` → tombol submit, seperti `<button type="submit">Simpan</button>` yang sudah kamu kenal dari form Tambah Buku/Anggota.

2.3 Wireframe yang Lebih Kompleks: Dashboard Petugas

```
+-----------------------------------------------------+
| SIMPUS-Mini     Beranda | Buku | Anggota | Peminjaman | (Nama Petugas) Logout |
|-----------------------------------------------------|
|  [Total Buku]   [Total Anggota]   [Sedang Dipinjam]    |
|                                                      |
|  Aksi Cepat:                                        |
|  [ + Peminjaman Baru ]   [ + Pengembalian ]         |
|                                                      |
|  Transaksi Terbaru                                  |
|  ------------------------------------------------    |
|  Anggota | Buku | Tgl Pinjam | Status               |
+-----------------------------------------------------+
```

Perhatikan baris navigasi paling atas: `Beranda | Buku | Anggota | Peminjaman` — ini terlihat sangat mirip navbar yang **sudah ada** di `index.html` saat ini, hanya **ditambah** satu menu baru ("Peminjaman") dan indikator status login di kanan (`(Nama Petugas) Logout`). Baris `[Total Buku] [Total Anggota] [Sedang Dipinjam]` juga persis menggambarkan kartu statistik yang **sudah kamu bangun** sejak jobsheet-02 memakai CSS Grid (lihat dokumentasi jobsheet-02).

Ini bukan kebetulan — wireframe Dashboard sengaja dirancang supaya **melanjutkan** elemen yang sudah ada (navbar, kartu statistik), ditambah bagian baru (`Aksi Cepat`, `Transaksi Terbaru`). Hubungan ini dibahas lebih detail di bab 5.

2.4 Kenapa Wireframe Sengaja Tidak Berwarna/Detail?

Wireframe sengaja dibuat **polos** (tanpa warna, tanpa font, tanpa ukuran presisi) supaya diskusi rancangan fokus ke **struktur dan alur** dulu — elemen apa saja yang perlu ada, di mana posisinya

secara garis besar — tanpa terjebak berdebat soal warna tombol atau jenis font di tahap yang masih sangat awal. Detail visual seperti itu **sudah punya** jawabannya** kalau nanti diimplementasikan, karena tinggal mengikuti** `style.css` yang sudah dibangun (dokumentasi jobsheet-02) — sesuai catatan di wireframe.md bagian "Konsistensi dengan Desain yang Sudah Berjalan".

Lanjut ke: User Flow: Peminjaman & Pengembalian

3. User Flow: Peminjaman & Pengembalian

Kalau wireframe (bab 2) menggambarkan **satu halaman**, **user flow** menggambarkan **urutan langkah** yang dilalui pengguna melintasi beberapa halaman/aksi untuk menyelesaikan satu tugas tertentu.

3.1 Apa itu User Flow?

User flow (alur pengguna) adalah diagram sederhana berbentuk kotak-kotak yang dihubungkan tanda panah `->`, menunjukkan **urutan** langkah demi langkah**. Setiap kotak mewakili satu layar, satu aksi, atau** satu keputusan yang dilakukan pengguna. Tujuannya: memastikan **tidak ada** langkah yang terlewat atau ambigu** sebelum fitur itu benar-benar dikoding.**

3.2 User Flow: Peminjaman Buku

```
[Petugas Login] -> [Dashboard] -> [Pilih menu "Peminjaman Baru"]
       -> [Pilih Anggota] -> [Pilih Buku (stok > 0)]
       -> [Simpan] -> [Stok buku berkurang 1] -> [Kembali ke Dashboard]
```

Mari telusuri kotak demi kotak:

1. `[Petugas Login]` — Petugas harus login lebih dulu (wireframe halaman ini sudah dibahas di bab 2 §2.1). Kotak ini menegaskan bahwa fitur peminjaman **tidak bisa diakses** Tamu** — dibahas lebih dalam soal aktor di** bab 4.
2. `[Dashboard]` — setelah login berhasil, Petugas mendarat di halaman Dashboard (wireframe-nya di bab 2 §2.3).
3. `[Pilih menu "Peminjaman Baru"]` — Petugas mengklik salah satu tombol "Aksi Cepat" yang ada di Dashboard.
4. `[Pilih Anggota]` dan `[Pilih Buku (stok > 0)]` — dua langkah pengisian form berurutan. Perhatikan catatan penting `(stok > 0)` — ini adalah **aturan bisnis** (business rule) yang

sengaja dituliskan di user flow: buku yang stoknya habis **tidak boleh muncul** sebagai pilihan. Menuliskan aturan ini di tahap rancangan memastikan developer tidak lupa menerapkannya nanti saat coding (ingat pentingnya merancang dulu dari bab 1 §1.2).

5. `[Simpan]` — Petugas menekan tombol submit (mirip `<button type="submit">` yang sudah kamu kenal dari dokumentasi jobsheet-01, hanya saja form ini nantinya **benar-benar akan diproses** ke database — beda dengan form Tambah Buku/Anggota yang di jobsheet-01 sampai jobsheet-03 memang sengaja belum diproses).
6. `[Stok buku berkurang 1]` — ini adalah **efek samping otomatis** di balik layar: begitu peminjaman disimpan, sistem harus mengurangi angka stok buku itu sendiri. Kotak ini **bukan** sebuah halaman, melainkan pengingat bahwa ada **logika program** yang perlu ditulis di balik tombol Simpan (baru akan diimplementasikan dengan JavaScript/backend di jobsheet-jobsheet mendatang — lihat README.md jobsheet ini).
7. `[Kembali ke Dashboard]` — alur ditutup dengan kembali ke titik awal, menunjukkan siklus selesai dan Petugas bisa mengulang alur ini untuk peminjaman berikutnya.

3.3 User Flow: Pengembalian Buku

```
[Dashboard] -> [Menu "Pengembalian"] -> [Cari transaksi aktif (anggota/buku)]
        -> [Tandai "Dikembalikan"] -> [Stok buku bertambah 1]
        -> [Kembali ke Dashboard]
```

Alur ini punya bentuk mirip, tapi perhatikan perbedaannya:

- Dimulai dari **mencari transaksi yang aktif** (peminjaman yang belum dikembalikan), bukan mengisi form data baru dari nol — masuk akal, karena mengembalikan buku berarti **mencocokkan** dengan data peminjaman yang sudah ada, bukan membuat data baru.
- Efek sampingnya adalah **kebalikan** dari alur peminjaman: stok **bertambah** 1, bukan berkurang.

Membandingkan dua user flow yang mirip strukturnya seperti ini adalah cara yang baik untuk memeriksa **konsistensi rancangan** — kalau kedua alur ini bentuknya sangat berbeda tanpa alasan jelas, itu bisa jadi tanda ada sesuatu yang perlu dipikirkan ulang.

3.4 Kenapa User Flow Ditulis Sebelum Ada Form-nya?

Sama seperti alasan di bab 1 §1.2: menuliskan urutan langkah ini **sebelum** ada satu baris kode form Peminjaman memastikan pertanyaan-pertanyaan penting sudah terjawab lebih dulu, misalnya:

- Apakah anggota bisa meminjam **lebih dari satu buku** sekaligus dalam satu transaksi? (Wireframe form Peminjaman di `docs/wireframe.md` menunjukkan satu buku per transaksi — ini keputusan rancangan yang sudah diambil sejak tahap ini.)
- Siapa saja yang boleh mengakses fitur ini? (Dijawab di bab 4.)
- Bagaimana urutan yang paling wajar dari sudut pandang Petugas yang benar-benar memakainya sehari-hari?

Lanjut ke: Aktor & Kaitannya dengan Otorisasi

4. Aktor & Kaitannya dengan Otorisasi

4.1 Apa itu "Aktor" dalam Rancangan UI/UX?

Aktor adalah istilah untuk **jenis pengguna** yang berinteraksi dengan sistem, dibedakan berdasarkan **apa yang boleh mereka lakukan**. `docs/wireframe.md` mendefinisikan 2 aktor untuk SIMPUS-Mini:

> - **Tamu**: hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
> - **Petugas**: login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

Mendefinisikan aktor di awal itu penting karena **halaman yang sama** persis** bisa punya arti berbeda tergantung siapa yang membukanya —** dan itu menentukan fitur apa saja yang perlu dibangun untuk masing-masing.

4.2 Aktor "Tamu" — Yang Sudah Bisa Kamu Coba Sekarang

Semua halaman yang **sudah ada** sejak jobsheet-01 sampai jobsheet-03 — Beranda, Daftar Buku, dan seterusnya — sebenarnya **mewakili sudut pandang aktor Tamu**: siapa pun bisa membukanya langsung tanpa perlu login apa pun. Coba perhatikan: halaman-halaman itu memang tidak punya elemen "logout" atau nama pengguna di mana pun — konsisten dengan definisi aktor Tamu yang "hanya bisa melihat katalog buku... tanpa login."

4.3 Aktor "Petugas" — Yang Baru Dirancang di Jobsheet Ini

Semua wireframe baru di `docs/wireframe.md` (Login, Dashboard, form Peminjaman/Pengembalian, Riwayat) adalah sudut pandang aktor **Petugas** — dan **semuanya** mensyaratkan login terlebih dulu. Ini terlihat jelas dari user flow yang sudah dibahas di bab 3: baik alur Peminjaman maupun

Pengembalian **selalu dimulai** dari kotak yang mengandaikan Petugas sudah masuk sistem (`[Petugas Login]` atau langsung `[Dashboard]`, yang hanya bisa dicapai setelah login).

4.4 Apa itu Otorisasi, dan Kenapa Belum Ada di Jobsheet Ini?

Otorisasi (*authorization*) adalah mekanisme sistem untuk **membatasi** siapa yang boleh melakukan/melihat apa — misalnya memastikan hanya Petugas yang login yang bisa membuka halaman Peminjaman, sementara Tamu yang mencoba membukanya akan dialihkan (redirect) ke halaman Login.

Perhatikan otorisasi ini **butuh kode program sungguhan** untuk bekerja (memeriksa status login, menyimpan sesi pengguna, dsb.) — sesuatu yang belum bisa dilakukan HTML/CSS statis seperti yang sudah kamu pelajari di jobsheet-01 sampai jobsheet-03. Inilah kenapa fitur Login dan Peminjaman **belum diimplementasikan** di jobsheet ini — sesuai catatan di README.md jobsheet ini, fitur ini baru akan dibangun mulai jobsheet-jobsheet mendatang dengan bantuan JavaScript, lalu PHP/PostgreSQL untuk mengelola data sungguhan di sisi server.

4.5 Kenapa Merancang Aktor Lebih Dulu Itu Penting?

Dengan mendefinisikan 2 aktor ini **sebelum** menulis kode Login, developer sudah punya jawaban jelas untuk pertanyaan-pertanyaan yang akan muncul saat coding nanti, misalnya:

- Halaman mana saja yang perlu dijaga dengan pengecekan login, dan mana yang tetap boleh diakses bebas? (Jawabannya sudah tersirat dari pembagian aktor: halaman katalog untuk Tamu tetap bebas, halaman transaksi untuk Petugas perlu dijaga.)
- Apa yang terjadi kalau Tamu mencoba mengakses URL halaman Dashboard secara langsung tanpa login? (Ini "edge case" yang perlu dipikirkan sejak tahap rancangan, meski implementasi teknisnya menyusul nanti.)

Lanjut ke: Keterhubungan dengan Kode yang Sudah Ada

5. Keterhubungan dengan Kode yang Sudah Ada

Bab ini menghubungkan rancangan wireframe di `docs/wireframe.md` dengan kode HTML/CSS yang **sudah benar-benar berjalan** sejak jobsheet-01 sampai jobsheet-03 — supaya kamu melihat bahwa rancangan ini bukan proyek terpisah, melainkan **kelanjutan langsung** dari yang sudah kamu bangun.

5.1 Catatan Resmi dari `wireframe.md`

Dokumen `wireframe.md` sendiri menutup dengan bagian "Konsistensi dengan Desain yang Sudah Berjalan":

> - Warna aksen, tipografi navbar, dan gaya tabel/kartu mengikuti `assets/css/style.css` yang sudah dibangun sejak Jobsheet 2-3.
> - Navbar akan ditambah menu **Peminjaman** dan indikator status login (nama petugas / tombol Logout) mulai implementasi di Jobsheet 10.
> - Edge case yang perlu ditangani saat implementasi: buku stok habis tidak boleh dipilih di form peminjaman; anggota dengan tunggakan terlambat divalidasi di Jobsheet 12 (tugas mandiri).

Mari bedah tiap poin ini satu per satu.

5.2 Warna & Gaya Tetap Sama — Tidak Perlu Membangun CSS dari Nol

Ingat dari dokumentasi jobsheet-02, kamu sudah membangun:

- Warna tema biru `#1d5b8a` untuk header, judul section, dan tombol submit (lihat dokumentasi jobsheet-02 §3.2).
- Gaya kartu putih dengan bayangan halus untuk `<section>` (lihat dokumentasi jobsheet-02 §5.3).
- Gaya tabel dengan header berwarna dan baris selang-seling (lihat dokumentasi jobsheet-02 §7).
- Gaya form dengan label tebal dan input rapi (lihat dokumentasi jobsheet-02 §8).

Semua wireframe halaman baru (Login, Dashboard, form Peminjaman/ Pengembalian) **sengaja dirancang mengikuti pola yang sama** — artinya saat halaman-halaman ini benar-benar dikoding nanti, kamu **tidak perlu** menulis ulang CSS dari nol**. Form Login misalnya, akan memakai pola** `<label>` + `<input>` yang **identik** dengan form Tambah Buku yang sudah kamu pelajari di dokumentasi jobsheet-01, sehingga otomatis mendapat gaya rapi dari `form label` dan `form input` di `style.css` tanpa perlu menulis CSS baru.

5.3 Navbar Akan Bertambah Menu — Tapi Pola CSS-nya Tidak Berubah

Ingat dari dokumentasi jobsheet-02 dan dokumentasi jobsheet-03, navbar dibangun dengan Flexbox (`header nav ul { display: flex; ... }`) yang menyusun **jumlah item menu berapa pun** secara otomatis sejajar horizontal (atau vertikal di layar mobile lewat checkbox hack). Artinya, menambahkan satu `<li><a>` baru untuk menu "Peminjaman" nanti di Jobsheet 10 **tidak memerlukan perubahan CSS sama sekali** — cukup menambah satu baris HTML, dan Flexbox akan otomatis menata ulang seluruh menu. Ini contoh nyata manfaat mendesain sistem CSS yang generik sejak awal (ingat catatan serupa soal selector generik di dokumentasi jobsheet-02 §6.7).

Begitu juga indikator status login (`(Nama Petugas) Logout`) yang terlihat di wireframe Dashboard — ini akan jadi elemen tambahan baru di dalam `<header>`, kemungkinan disusun sejajar dengan `<h1>` dan `<nav>` memakai `justify-content: space-between` yang sama seperti yang membuat `<h1>` dan `<nav>` sejajar sekarang (lihat dokumentasi jobsheet-02 §4.4).

5.4 Kartu Statistik yang Sama, Konteks yang Beda

Wireframe Dashboard Petugas menampilkan baris `[Total Buku] [Total Anggota] [Sedang Dipinjam]` — bandingkan dengan kartu statistik yang **sudah ada dan berfungsi** di `index.html` saat ini (lihat penjelasan lengkapnya di dokumentasi jobsheet-01 dan cara stylingnya dengan CSS Grid di dokumentasi jobsheet-02). Ini bukan kebetulan — rancangan Dashboard Petugas **menggunakan ulang** komponen kartu statistik yang sama persis, hanya dalam konteks halaman yang berbeda (khusus Petugas yang sudah login, bukan Beranda publik).

5.5 Edge Case yang Sudah Dicatat Sejak Sekarang

Dua "edge case" (kasus khusus/pengecualian) sudah dicatat di `wireframe.md`:

1. **Buku stok habis tidak boleh dipilih** di form Peminjaman — ini sudah muncul juga di user flow (bab 3 §3.2, catatan `(stok > 0)`). Data stok ini **sudah ada** sejak jobsheet-01: lihat kolom "Stok" di tabel Daftar Buku — nantinya tinggal ditambah logika pengecekan `stok > 0` saat menampilkan pilihan buku di form Peminjaman.
2. **Anggota dengan tunggakan terlambat** — kasus ini ditandai untuk ditangani di "Jobsheet 12 (tugas mandiri)", menunjukkan bahwa tidak semua detail perlu diselesaikan **sekaligus**; mencatatnya di tahap rancangan memastikan kasus ini tidak terlupakan meski implementasinya ditunda ke jobsheet lain.

Mencatat edge case seperti ini di tahap wireframe — jauh sebelum menulis kode penanganannya — adalah kebiasaan baik yang mencegah "kejutan" di tengah proses coding nanti.

Lanjut ke: Rangkuman & Latihan Lanjutan

6. Rangkuman & Latihan Lanjutan

6.1 Rangkuman Keseluruhan Jobsheet 4

Bagian	Konsep yang Dipelajari
Konsep Dasar UI/UX	Beda UI vs UX, alasan merancang sebelum coding
Cara Membaca Wireframe	Konvensi simbol ASCII wireframe, menerjemahkannya ke elemen HTML
User Flow	Diagram alur langkah demi langkah, mencatat aturan bisnis sejak tahap rancangan
Aktor & Otorisasi	Konsep aktor (Tamu vs Petugas), pengantar otorisasi
Keterhubungan dengan Kode	Bagaimana rancangan baru memakai ulang CSS/pola yang sudah dibangun sejak jobsheet-02/03

6.2 Konsep Inti yang Perlu Diingat

1. **Tidak semua jobsheet harus menambah kode.** Merancang dulu sebelum coding adalah bagian sah dan penting dari proses pengembangan software, bukan langkah yang bisa dilewati begitu saja (bab 1).
2. **Wireframe menjawab "apa isinya", user flow menjawab "bagaimana** urutannya"** — keduanya saling melengkapi, bukan saling** menggantikan (bab 2, bab 3).
3. **Mendefinisikan aktor di awal** membantu menentukan halaman mana yang perlu dijaga otorisasinya, jauh sebelum kode pengecekan login itu sendiri ditulis (bab 4).
4. **Rancangan yang baik memakai ulang** apa yang sudah ada (CSS, pola komponen) alih-alih membangun dari nol — terlihat dari bagaimana wireframe Dashboard mengadopsi kartu statistik dan navbar yang sudah kamu bangun sejak jobsheet-02 (bab 5).
5. **Mencatat edge case sejak tahap rancangan** (stok buku habis, anggota bertunggakan) mencegah kasus-kasus itu terlupakan saat coding sungguhan nanti (bab 5 §5.5).

6.3 Cara "Mencoba" Jobsheet Ini

Karena tidak ada kode baru untuk dijalankan di browser, cara terbaik mempraktikkan jobsheet ini adalah **membaca ulang wireframe sambil** membayangkan dirimu sebagai Petugas** yang menjalankan setiap langkah:**

1. Buka `docs/wireframe.md` dan baca tiap wireframe sambil membayangkan bagaimana tampilannya kalau digambar sungguhan mengikuti `style.css` yang sudah ada.
2. Ikuti user flow Peminjaman (bab 3 §3.2) langkah demi langkah sambil membayangkan halaman mana yang terbuka di tiap kotak.
3. Bandingkan navbar dan kartu statistik di `index.html` yang sudah berjalan dengan wireframe Dashboard Petugas — temukan sendiri bagian mana yang sama persis dan mana yang baru.

6.4 Ide Latihan Tambahan (Opsional)

1. **Gambar wireframe halaman baru** memakai konvensi ASCII yang sama (bab 2 §2.2) — misalnya halaman "Registrasi Anggota Baru" untuk aktor Tamu yang ingin jadi anggota perpustakaan.
2. **Buat user flow baru** untuk skenario yang belum digambarkan di `wireframe.md`, misalnya: "Petugas mencari anggota yang tunggakannya sudah lewat jatuh tempo."
3. **Identifikasi edge case tambahan** yang mungkin belum tercatat, contoh: apa yang terjadi kalau Petugas mencoba meminjamkan buku yang sama ke anggota yang sama dua kali berturut-turut?
4. **Coba implementasikan wireframe Login sebagai HTML statis** (tanpa logika login sungguhan, mirip form Tambah Buku yang belum diproses di jobsheet-01) sebagai latihan menerjemahkan wireframe ke kode nyata — gunakan pola `<label>` + `<input>` yang sudah kamu kuasai dari dokumentasi jobsheet-01, ditambah satu `<input type="password">` baru untuk field Password.

Kalau ada bagian yang masih membingungkan, coba baca ulang bab 1 — konsep "kenapa merancang dulu" adalah fondasi yang menjelaskan alasan di balik seluruh isi jobsheet ini.